Explicabilidade e Transparência em Agentes Baseados em LLM ao Longo do Ciclo de Vida de Desenvolvimento de *Software* (SDLC): Um Mapeamento Sistemático da Literatura

Jordane Gabriella S. O. Pires[1], Matheus Augusto F. Medeiros[1], Matheus Vieira Mendes Pacheco[1], Davi Duarte Neco[1]

[1]Instituto de Informática – Engenharia de *Software* – Universidade Federal de Goiás (UFG)

{davi.neco, jordanegabriela, matheus augusto, matheus.pacheco}@discente. ufg.br

Abstract. *This paper presents a systematic mapping study on explainability and transparency approaches for LLM-based agents across the Software Development Life Cycle (SDLC). It investigates the techniques employed, their distribution across SDLC phases, their impact on developer trust and debugging, and how explanation quality is evaluated. Following a systematic protocol over IEEE Xplore, ACM Digital Library and Google Scholar, 16 primary studies (2022–2026) were analyzed. Results show a predominance of natural-language justifications, a concentration on implementation and debugging/maintenance activities, and scarce assessment of explanation faithfulness, revealing gaps for early SDLC phases and autonomous agents.*

Keywords: *Large Language Models, LLM Agents, Explainable Artificial Intelligence, Explainability, Transparency, Software Engineering, Systematic Mapping Study.*

Resumo. *Este artigo apresenta um mapeamento sistemático sobre explicabilidade e transparência em agentes baseados em LLM ao longo do ciclo de vida de desenvolvimento de* software *(SDLC). O trabalho investiga as técnicas empregadas, sua distribuição entre as fases do SDLC, seus impactos na confiança e na depuração e como a qualidade das explicações é avaliada. Seguindo um protocolo sistemático nas bases IEEE Xplore, ACM Digital Library e Google Scholar, foram analisados 16 estudos primários (2022–2026). Os resultados mostram o predomínio de justificativas em linguagem natural, a concentração em atividades de implementação e de depuração e manutenção e a escassa avaliação da fidelidade das explicações, revelando lacunas nas fases iniciais do SDLC e para agentes autônomos.*

1. Introdução

A crescente adoção de *Large Language Models* (LLMs) tem ampliado as possibilidades de aplicação da Inteligência Artificial em diferentes atividades da Engenharia de *Software*. Além do uso desses modelos como ferramentas de apoio pontual, observa-se o desenvolvimento de agentes baseados em LLM capazes de executar tarefas com diferentes níveis de autonomia, incluindo análise de requisitos, geração e revisão de código, elaboração de testes, identificação de falhas e apoio à manutenção de sistemas. Nesse cenário, a atuação desses agentes passa a abranger diferentes etapas do ciclo de vida de desenvolvimento de *software* (*Software Development Life Cycle*, SDLC).

O aumento da autonomia desses sistemas, entretanto, também amplia a necessidade de compreender e acompanhar as decisões tomadas durante sua execução. Em atividades de Engenharia de *Software*, não é suficiente que um agente produza uma resposta aparentemente adequada: em determinados contextos, o engenheiro também precisa identificar quais informações fundamentaram uma decisão, quais etapas foram executadas e quais elementos contribuíram para determinado resultado. Essa necessidade torna especialmente relevantes aspectos como transparência, rastreabilidade e explicabilidade, principalmente diante da possibilidade de geração de informações incorretas ou não fundamentadas por LLMs.

Embora existam revisões sobre o uso de LLMs na Engenharia de *Software* [Hou et al. 2024, He et al. 2025] e sobre explicabilidade em modelos de aprendizado de máquina aplicados à área [Mohammadkhani et al. 2023, Cao et al. 2024], não há um panorama que relacione as técnicas de explicabilidade de agentes baseados em LLM às fases do SDLC e sintetize como sua eficácia é avaliada.

Diante disso, este artigo apresenta os resultados iniciais de um mapeamento sistemático da literatura, conduzido segundo as diretrizes de Petersen et al. [2015] e Kitchenham and Charters [2007], que analisa 16 estudos primários publicados entre 2022 e 2026. As contribuições são: (i) uma classificação das técnicas de explicabilidade em cinco categorias; (ii) a distribuição dessas técnicas pelas fases do SDLC; (iii) uma síntese das evidências sobre confiança, depuração e excesso de confiança; e (iv) um levantamento dos métodos de avaliação das explicações.

2. Fundamentação teórica

2.1. Agentes baseados em LLM na Engenharia de *Software*

Um agente baseado em LLM é um sistema em que o modelo de linguagem atua como componente de raciocínio que planeja, executa ações por meio de ferramentas (como edição de arquivos e execução de testes) e observa os resultados dessas ações para decidir o passo seguinte [Yao et al. 2023]. Na Engenharia de *Software*, surgiram agentes capazes de resolver *issues* reais de repositórios, avaliados em *benchmarks* como o SWE-bench. Com menor supervisão humana, cresce a preocupação com a confiança necessária para aceitar o que produzem [Roychoudhury et al. 2025]. Neste mapeamento, consideram-se tanto agentes autônomos quanto LLMs usados como assistentes, pois os mecanismos de explicação são, em grande parte, os mesmos.

2.2. Explicabilidade, transparência e interpretabilidade

Os três termos são frequentemente usados de forma intercambiável, mas têm significados distintos. Interpretabilidade refere-se à capacidade de apresentar o comportamento de um modelo em termos compreensíveis a um humano; explicabilidade diz respeito às explicações produzidas, durante ou após a execução, para justificar uma saída específica; e transparência refere-se à visibilidade sobre o funcionamento do sistema e sobre o processo que levou a um resultado [Barredo Arrieta et al. 2020]. Em agentes, a transparência inclui a rastreabilidade da trajetória: a sequência de pensamentos, chamadas de ferramentas e observações.

Em LLMs, as explicações incluem raciocínio verbalizado, como *chain-of-thought* (CoT) [Wei et al. 2022]; atribuição de importância à entrada, como valores de Shapley

[Lundberg and Lee 2017]; proveniência das informações recuperadas em geração aumentada por recuperação (RAG) [Lewis et al. 2020]; e sinais de incerteza. Uma questão central é a fidelidade (*faithfulness*), isto é, o quanto a explicação reflete o processo que de fato produziu a saída [Jacovi and Goldberg 2020]. Turpin et al. [2023] mostram que explicações em CoT podem ser plausíveis e não fiéis.

2.3. Trabalhos relacionados

Mohammadkhani et al. [2023] revisaram 24 estudos sobre explicabilidade aplicada à Engenharia de *Software*, concentrados em predição de defeitos, e Cao et al. [2024] ampliaram o escopo para 108 estudos baseados em aprendizado de máquina e profundo. Ambas tratam majoritariamente de modelos anteriores aos LLMs e não consideram agentes. Já Hou et al. [2024] e He et al. [2025] mapeiam o uso de LLMs e de sistemas multiagentes na área, mas abordam a explicabilidade apenas como desafio em aberto. Este trabalho situa-se na interseção desses dois conjuntos.

3. Metodologia

O estudo segue as diretrizes de Petersen et al. [2015] para mapeamentos sistemáticos, complementadas pelas recomendações de Kitchenham and Charters [2007] para a definição do protocolo.

3.1. Objetivo do mapeamento

Este estudo tem como objetivo mapear sistematicamente as abordagens de explicabilidade e transparência aplicadas a agentes baseados em *Large Language Models* (LLMs) no contexto da Engenharia de *Software*, identificando as técnicas utilizadas, sua aplicação nas diferentes fases do SDLC e os métodos empregados para avaliar sua eficácia e utilidade para desenvolvedores.

3.2. Questões de pesquisa

- **RQ1 (Taxonomia e técnicas de XAI):** Quais são as técnicas de explicabilidade (ex.: raciocínio verbalizado/CoT, proveniência de RAG, árvores de decisão de agentes, saliência de código/*tokens*) mais utilizadas para tornar transparentes as decisões de agentes LLM?
- **RQ2 (Mapeamento ao longo do SDLC):** Como as abordagens de explicabilidade variam entre as diferentes fases do ciclo de vida do *software* (ex.: explicação da escolha de arquitetura vs. justificativa para a geração de casos de teste ou correções de código)?
- **RQ3 (Eficácia e confiança do desenvolvedor):** Qual é o impacto documentado da explicabilidade dos agentes na tomada de decisão humana, na facilidade de depuração e na mitigação do excesso de confiança (*over-reliance*) do engenheiro no código gerado?
- **RQ4 (Métricas e métodos de avaliação de XAI):** Como os estudos da literatura avaliam empiricamente a qualidade, a fidelidade e a utilidade das explicações fornecidas pelos agentes aos engenheiros de *software* (ex.: métricas automáticas, testes de uso com programadores, auditoria humana)?

3.3. Estratégia de busca

A *string* de busca foi construída a partir de três grupos de termos derivados das questões de pesquisa: (i) a tecnologia investigada (LLMs e agentes de LLM); (ii) o atributo de interesse (explicabilidade, transparência e interpretabilidade); e (iii) o domínio de aplicação (Engenharia e desenvolvimento de *software*). Os sinônimos de cada grupo foram combinados pelo operador OR e os grupos, pelo operador AND, resultando na seguinte *string*:

```
("large language model" OR "LLM agent" OR "LLM agents") AND
(explainability OR transparency OR interpretability) AND
("software engineering" OR "software development")
```

A *string* foi adaptada à sintaxe de cada base e aplicada a todos os campos disponíveis, sem restrição de período. Termos de técnicas específicas (ex.: *chain-of-thought*) foram omitidos para não enviesar a busca. Para mitigar a cobertura limitada da *string*, a busca automática foi complementada por *snowballing* para trás e para frente [Wohlin 2014] a partir dos estudos incluídos.

3.4. Fontes de busca

Foram utilizadas a *IEEE Xplore* e a *ACM Digital Library*, que indexam os principais veículos de Engenharia de *Software*, e o *Google Scholar*, que abrange também *preprints*, relevantes em uma área de evolução rápida. Como os resultados são ordenados por relevância e o volume é elevado, a triagem considerou os 10 primeiros resultados na *IEEE Xplore* e os [N] primeiros na *ACM Digital Library* e no *Google Scholar*.

3.5. Critérios de inclusão e exclusão

Foram incluídos os estudos que atendem a todos os critérios de inclusão (CI) e a nenhum dos critérios de exclusão (CE):

CI1 – O estudo aborda LLMs ou agentes de LLMs.

CI2 – O LLM/agente é aplicado a uma atividade de Engenharia de *Software* ou etapa do SDLC, como requisitos, testes, arquitetura, codificação, DevOps ou manutenção.

CI3 – O estudo aborda algum mecanismo de explicabilidade, transparência, interpretabilidade, rastreabilidade ou proveniência relacionado ao comportamento/resultado do LLM ou agente.

CI4 – O estudo apresenta informação suficiente para responder a pelo menos uma das RQs do mapeamento.

CI5 – O texto completo está disponível para análise.

CI6 – O estudo está escrito em inglês ou português.

CE1 – Estudo duplicado entre as bases.

CE2 – Utiliza LLM, mas fora do contexto de Engenharia de *Software*.

CE3 – Trata de Engenharia de *Software* e LLM, mas não aborda explicabilidade/transparência.

CE4 – Trata de XAI/explicabilidade, mas não envolve LLMs ou agentes baseados em LLMs.

CE5 – O LLM é usado apenas como instrumento auxiliar da pesquisa, sem que sua explicabilidade/transparência seja objeto do estudo.

CE6 – Discute requisitos de explicabilidade de outros sistemas de IA, sem investigar a explicabilidade/transparência do LLM/agente atuando no SDLC.

CE7 – Não possui texto completo acessível ou não apresenta informações suficientes para análise.

3.6. Processo de seleção dos estudos

A seleção foi realizada em quatro etapas: (1) reunião dos resultados e remoção de duplicados (CE1); (2) aplicação dos critérios a título, resumo e palavras-chave, com os casos duvidosos encaminhados à etapa seguinte; (3) leitura completa e reaplicação dos critérios, com atenção a CE5 e CE6, que distinguem os estudos em que a explicabilidade do LLM é objeto de investigação daqueles em que o LLM é apenas instrumento; e (4) *snowballing* a partir dos estudos incluídos. Por exemplo, Obaidi et al. [2025] usam LLMs para gerar requisitos de explicabilidade do *software* final, e não do modelo, e foram excluídos por CE6. Os casos de divergência entre os pesquisadores foram discutidos até se chegar a um consenso.

A busca retornou [X] estudos na *IEEE Xplore*, [Y] na *ACM Digital Library* e [Z] no *Google Scholar*. Após a remoção de duplicatas, a triagem por título e resumo, a leitura completa e o *snowballing*, 16 estudos foram incluídos (Tabela 1).

3.7. Extração e classificação dos dados

Para cada estudo, um formulário registrou: identificação (autores, ano, veículo); nível de autonomia (assistente ou agente); técnica de explicabilidade (RQ1); fase do SDLC (RQ2); efeitos sobre confiança, decisão, depuração e excesso de confiança (RQ3); e método de avaliação, com métricas e número de participantes (RQ4).

As técnicas foram classificadas por codificação aberta seguida de agrupamento, partindo dos exemplos citados na RQ1, o que resultou em cinco categorias: **T1** – justificativa em linguagem natural (raciocínio verbalizado, CoT ou explicação textual da saída); **T2** – atribuição pós-hoc (importância de *tokens*, trechos ou características da entrada); **T3** – sinalização de incerteza e confiança; **T4** – rastreamento de trajetórias do agente; e **T5** – elicitação de necessidades de explicação. Cada estudo foi associado à técnica predominante. As fases do SDLC seguem Hou et al. [2024], acrescidas de revisão de código e operação.

4. Resultados iniciais

Os 16 estudos incluídos (Tabela 1) foram publicados entre 2022 e 2026, com crescimento a partir de 2024 (1 em 2022, 1 em 2023, 4 em 2024, 7 em 2025 e 3 em 2026). Oito foram publicados em veículos com revisão por pares e oito são *preprints*.

Tabela 1. Estudos primários incluídos no mapeamento

ID	Estudo	Fase / atividade	Téc.	Avaliação das explicações
E01	Sun et al. [2022]	Implementação (geração)	T5	Workshops com 43 engenheiros
E02	Rodriguez et al. [2023]	Requisitos (rastreabilidade)	T1	Precisão/revocação; qualitativa
E03	Chen et al. [2024]	Operação (incidentes em nuvem)	T1	Micro/Macro-F1 (653 incidentes)
E04	Widyasari et al. [2024]	Depuração (localização de falhas)	T1	BLEURT; avaliação humana (10)
E05	Palacio et al. [2024]†	Implementação (geração)	T2	Questionário (24); análise causal
E06	Virk et al. [2024]†	Manutenção (sumarização)	T3	Calibração (Brier score, ECE)
E07	Kang et al. [2025]	Depuração e reparo	T1	Estudo com 20 participantes

ID	Estudo	Fase / atividade	Téc.	Avaliação das explicações
E08	Vasconcelos et al. [2025]	Implementação (completação)	T3	Experimento controlado (30)
E09	Zhou et al. [2025]†	Arquitetura (design rationale)	T1	P/R/F1 vs. especialistas; qualitativa
E10	Ceka et al. [2025]†	Manutenção (issues)	T4	Trajetórias de 5 agentes
E11	Bouzenia and Pradel [2025]	Manutenção (issues)	T4	120 trajetórias anotadas
E12	Vitale et al. [2025]†	Implementação (geração)	T2	Fidelidade; questionário (37)
E13	Khati et al. [2025]†	Implementação e testes	T2	Entropia; estudo com usuários (37)
E14	Akbarova et al. [2026]	Testes (testes de fronteira)	T1	Questionário (27); entrevistas (6)
E15	Heander et al. [2026]†	Revisão de código	T3	Design participativo; survey (43)
E16	Gao et al. [2026]†	Revisão de código	T1	Experimento within-subjects (34)

† *Preprint* sem revisão por pares até a data da busca. T1: justificativa em linguagem natural; T2: atribuição pós-hoc; T3: incerteza e confiança; T4: trajetórias do agente; T5: necessidades de explicação.

4.1. RQ1: Técnicas de explicabilidade

A justificativa em linguagem natural (T1) é a técnica mais frequente (7 estudos), aplicada como raciocínio passo a passo para localizar falhas [Widyasari et al. 2024], ciclos de hipótese e experimento na depuração [Kang et al. 2025], justificativa de *links* de rastreabilidade [Rodriguez et al. 2023], argumentos de *design rationale* [Zhou et al. 2025] e explicações de testes de fronteira [Akbarova et al. 2026], de causas-raiz de incidentes [Chen et al. 2024] e de comentários de revisão [Gao et al. 2026]. A atribuição pós-hoc (T2, 3 estudos) inclui valores de Shapley [Vitale et al. 2025], confiança por *token* alinhada à árvore sintática [Palacio et al. 2024] e agregação global de explicações por *token* [Khati et al. 2025]. A sinalização de incerteza (T3, 3 estudos) aparece no destaque de *tokens* propensos a edição [Vasconcelos et al. 2025], na calibração da confiança de resumos de código [Virk et al. 2024] e em sinais de risco por linha e arquivo [Heander et al. 2026]. O rastreamento de trajetórias (T4, 2 estudos) analisa sequências de pensamento, ação e resultado de agentes [Bouzenia and Pradel 2025, Ceka et al. 2025], e Sun et al. [2022] (T5) elicitaram necessidades de explicação com 43 engenheiros.

Duas técnicas citadas na RQ1 não foram encontradas como objeto principal de nenhum estudo: a proveniência em RAG e as árvores de decisão de agentes. Embora Chen et al. [2024] e Virk et al. [2024] recuperem exemplos para o *prompt*, nenhum mostra ao usuário a origem das informações usadas. Além disso, apenas 5 dos 16 estudos (E07, E09, E10, E11 e E15) tratam de agentes com autonomia de múltiplos passos; os demais investigam LLMs usados como assistentes.

4.2. RQ2: Explicabilidade ao longo do SDLC

A Figura 1 relaciona as técnicas às fases do SDLC. Implementação e depuração/manutenção concentram 10 dos 16 estudos (5 cada), seguidas pela revisão de código (2); requisitos, arquitetura, testes e operação têm um estudo cada. A técnica empregada varia com a fase: a atribuição pós-hoc (T2) aparece apenas na implementação, em que a explicação pode ser ancorada em trechos da entrada; já nas fases em que a saída é uma decisão ou julgamento predomina a justificativa em linguagem natural (T1).

O objeto da explicação também muda: na arquitetura, explica-se por que uma decisão foi tomada, com argumentos sobre *trade-offs* [Zhou et al. 2025]; nos testes, por que um par de entrada e saída é uma fronteira [Akbarova et al. 2026]; e na depuração, por que uma linha é suspeita ou um *patch* corrige a falha [Widyasari et al. 2024, Kang et al. 2025]. O rastreamento de trajetórias (T4) restringe-se à resolução de *issues*.

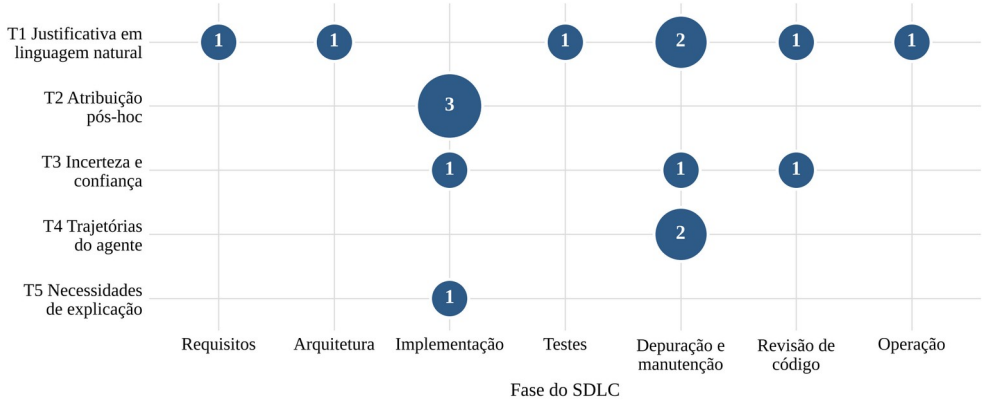


Figura 1. Distribuição dos estudos por técnica de explicabilidade e fase do SDLC (n = 16)

4.3. RQ3: Impacto na confiança, na decisão e na depuração

Os estudos com desenvolvedores indicam efeitos positivos, mas não uniformes. Na depuração, as explicações do AutoSD aumentaram a acurácia ao julgar a corretude de *patches* em cinco de seis *bugs*, sem aumentar o tempo, e 70% dos 20 participantes declararam desejar explicações [Kang et al. 2025]. Na revisão de código, Gao et al. [2026] observaram, com 34 participantes, que explicações completas geraram a maior confiança percebida, mas a maior concordância com a IA ocorreu com explicações moderadas, sugerindo que explicações detalhadas levam o desenvolvedor a questionar mais. Na completação de código, destacar os *tokens* com maior probabilidade de edição levou a edições mais direcionadas e a tarefas mais rápidas, enquanto destacar *tokens* de baixa probabilidade de geração não trouxe benefício [Vasconcelos et al. 2025].

Quanto ao excesso de confiança, as evidências ainda são exploratórias, mas apontam riscos. Vasconcelos et al. [2025] observaram participantes que editaram apenas os trechos destacados e ignoraram um caso de borda, lendo a ausência de destaque como sinal de corretude. Akbarova et al. [2026] relatam que uma única explicação alucinada reduziu fortemente as avaliações dos participantes. Heander et al. [2026] tratam a revisão de mudanças geradas por agentes como um problema de calibração de confiança.

4.4. RQ4: Métodos de avaliação das explicações

Os estudos combinam três tipos de avaliação (Tabela 1): estudos com desenvolvedores (10 estudos), métricas automáticas (8) e análise qualitativa de artefatos ou trajetórias (4). Entre os estudos com usuários predominam questionários Likert sobre clareza, utilidade e confiança; experimentos controlados com medidas de desempenho são menos frequentes [Kang et al. 2025, Vasconcelos et al. 2025, Gao et al. 2026], e as amostras variam de 10 a 43 participantes. As métricas automáticas incluem similaridade textual com explicações de referência [Widyasari et al. 2024], precisão e revocação frente a especialistas [Zhou et al. 2025], calibração [Virk et al. 2024] e entropia [Khati et al. 2025].

Apenas dois estudos avaliaram explicitamente a fidelidade das explicações: Vitale et al. [2025], por meio de perturbação da entrada, e Palacio et al. [2024], por inferência causal. Rodriguez et al. [2023] declaram essa verificação fora de seu escopo. Assim, nenhum dos sete estudos de T1 avaliou a fidelidade das justificativas, embora essa seja a técnica mais usada e haja evidência de explicações plausíveis, porém não fiéis [Turpin et al. 2023].

5. Ameaças à validade

As ameaças foram analisadas segundo as categorias propostas por Ampatzoglou et al. [2019] para estudos secundários. Quanto à **identificação dos estudos**, a *string* não inclui variações como *explainable*, *explanation*, *rationale* ou *trajectory*, e a triagem nas bases foi limitada aos primeiros resultados por relevância, o que pode ter omitido estudos; o risco foi mitigado pelo *snowballing* e pelo uso de três bases. Quanto à **seleção**, a distinção entre CE5 e CE6 envolve julgamento, mitigado pela leitura completa dos casos duvidosos e pelo consenso entre os pesquisadores.

Quanto à **extração e classificação**, as categorias T1–T5 foram definidas pelos autores e o registro apenas da técnica predominante pode subestimar combinações. Metade dos estudos são *preprints*, cujos resultados podem mudar após a revisão por pares; foram mantidos por refletirem o estado atual de uma área de evolução rápida e estão identificados na Tabela 1. Por fim, quanto à **validade das conclusões**, os 16 estudos correspondem a resultados iniciais, e as frequências relatadas são indicativas, não representativas.

6. Considerações finais

Este artigo apresentou os resultados iniciais de um mapeamento sistemático sobre explicabilidade e transparência em agentes baseados em LLM ao longo do SDLC. A análise de 16 estudos mostrou que: (i) a justificativa em linguagem natural é a técnica mais utilizada; (ii) as pesquisas se concentram em implementação e em depuração e manutenção; (iii) as explicações tendem a melhorar a acurácia do julgamento e a calibração da confiança, mas podem induzir dependência dos sinais exibidos; e (iv) a avaliação apoia-se em percepções de usuários e métricas de similaridade, com pouca atenção à fidelidade.

Como lacunas, destacam-se a ausência de estudos sobre proveniência em RAG e árvores de decisão de agentes, a escassez de explicações para agentes autônomos fora da resolução de *issues*, a falta de avaliação da fidelidade das justificativas e a inexistência de estudos longitudinais sobre excesso de confiança em contexto industrial. Como trabalhos futuros, pretende-se ampliar a busca para bases como Scopus, avaliar a qualidade dos estudos primários e comparar empiricamente técnicas de explicação para agentes em diferentes fases do SDLC.

Referências

Akbarova, S., Dobslaw, F., and Feldt, R. (2026). Understanding on the edge: LLM-generated boundary test explanations. In *Proceedings of the IEEE/ACM International Conference on Automation of Software Test (AST)*.

Ampatzoglou, A., Bibi, S., Avgeriou, P., et al. (2019). Identifying, categorizing and mitigating threats to validity in software engineering secondary studies. *Information and Software Technology*, 106:201–230.

Barredo Arrieta, A., Díaz-Rodríguez, N., Del Ser, J., et al. (2020). Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58:82–115.

Bouzenia, I. and Pradel, M. (2025). Understanding software engineering agents: A study of thought-action-result trajectories. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

Cao, S., Sun, X., Widyasari, R., et al. (2024). A systematic literature review on explainability for machine/deep learning-based software engineering research. arXiv:2401.14617.

Ceka, I., Pujar, S., Ramji, S., et al. (2025). Understanding software engineering agents through the lens of traceability: An empirical study. arXiv:2506.08311.

Chen, Y., Xie, H., Ma, M., et al. (2024). Automatic root cause analysis via large language models for cloud incidents. In *Proceedings of the 19th European Conference on Computer Systems (EuroSys)*. ACM.

Gao, Z., Muñoz Barón, M., Habiba, U., et al. (2026). Evaluating the impact of explainable AI on trust in AI-assisted code review. arXiv:2607.24601.

He, J., Treude, C., and Lo, D. (2025). LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. *ACM Transactions on Software Engineering and Methodology*, 34(5).

Heander, L. G., Sergeyuk, A., Zakharov, I., et al. (2026). Trust-calibrated code review: A participatory design study of review workflows for LLM-generated multi-file changes. arXiv:2606.01969.

Hou, X., Zhao, Y., Liu, Y., et al. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8).

Jacovi, A. and Goldberg, Y. (2020). Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness? In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4198–4205.

Kang, S., Chen, B., Yoo, S., and Lou, J.-G. (2025). Explainable automated debugging via large language model-driven scientific debugging. *Empirical Software Engineering*, 30(2):45.

Khati, D., Rodriguez-Cardenas, D., Palacio, D. N., et al. (2025). Enabling global, human-centered explanations for LLMs: From tokens to interpretable code and test generation. arXiv:2503.16771.

Kitchenham, B. and Charters, S. (2007). Guidelines for performing systematic literature reviews in software engineering. Technical Report EBSE-2007-01, Keele University and Durham University.

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33.

Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.

Mohammadkhani, A. H., Bommi, N. S., Daboussi, M., et al. (2023). A systematic literature review of explainable AI for software engineering. arXiv:2302.06065.

Obaidi, M., Fischbach, J., Droste, J., et al. (2025). Automatic generation of explainability requirements and software explanations from user reviews. arXiv:2507.07344.

Palacio, D. N., Rodriguez-Cardenas, D., Velasco, A., et al. (2024). Towards more trustworthy and interpretable LLMs for code through syntax-grounded explanations. arXiv:2407.08983.

Petersen, K., Vakkalanka, S., and Kuzniarz, L. (2015). Guidelines for conducting systematic mapping studies in software engineering: An update. *Information and Software Technology*, 64:1–18.

Rodriguez, A. D., Dearstyne, K. R., and Cleland-Huang, J. (2023). Prompts matter: Insights and strategies for prompt engineering in automated software traceability. In *IEEE 31st International Requirements Engineering Conference Workshops (REW)*.

Roychoudhury, A., Pasareanu, C., Pradel, M., and Ray, B. (2025). Agentic AI software engineers: Programming with trust. *Communications of the ACM*.

Sun, J., Liao, Q. V., Muller, M., et al. (2022). Investigating explainability of generative AI for code through scenario-based design. In *Proceedings of the 27th International Conference on Intelligent User Interfaces (IUI)*, pages 212–228. ACM.

Turpin, M., Michael, J., Perez, E., and Bowman, S. R. (2023). Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.

Vasconcelos, H., Bansal, G., Fourney, A., et al. (2025). Generation probabilities are not enough: Uncertainty highlighting in AI code completions. *ACM Transactions on Computer-Human Interaction*, 32(1).

Virk, Y., Devanbu, P., and Ahmed, T. (2024). Enhancing trust in LLM-generated code summaries with calibrated confidence scores. arXiv:2404.19318.

Vitale, A., Nguyen, K.-N., Poshyvanyk, D., et al. (2025). Toward explaining large language models in software engineering tasks. arXiv:2512.20328.

Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35.

Widyasari, R., Ang, J. W., Nguyen, T. G., et al. (2024). Demystifying faulty code: Step-by-step reasoning for explainable fault localization. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 568–579.

Wohlin, C. (2014). Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering (EASE)*. ACM.

Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Zhou, X., Li, R., Liang, P., et al. (2025). Using LLMs in generating design rationale for software architecture decisions. arXiv:2504.20781.